University of Birmingham

Ransomware (Malware)

Forensics, Malware and Pen Testing

Group 02
Benjamin Akyen, Rico Wedig, Cédric Lüchinger

22 March 2024

Contents

1 Introduction **2**

2 Reverse Engineering **2**
2.1 Start Up 2
2.2 Code analysing 3

3 Targeted Files and Directories **4**

4 Recovery Key & Tool **5**

References **6**

A Appendix **6**

1 Introduction

This assignment was conducted by group 2 for the Forensic, Malware, and Pen Testing module of the University of Birmingham. The goal of this report is to analyze the received ransomware and decrypt the files that were previously encrypted by said ransomware.

2 Reverse Engineering

2.1 Start Up

In the beginning, we look at the malware stored in the folder. The first substantial information is that it is a DOS/portable executable (PE), meaning this malware only runs on Windows machines. After gathering that information, we analyzed the ransomware in Ghidra using our Arch Linux VM, where we found some interesting information. First, we can see that the file is relocatable, thus being able to be located in different memory addresses, which complicates the detection of malware.

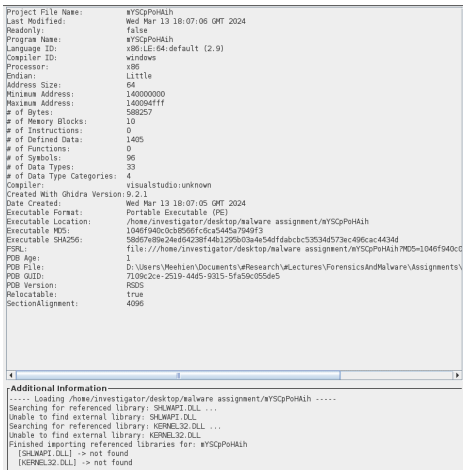

Figure 1: Ghidra Malware import

In the bottom section of Figure 1 in the "additional information" section, it tries to load two different libraries. "KERNEL32.DLL" is a core part of the OS. It performs memory management operations and input/output operations. Additionally, it wants to load the "SHLWAPI.DLL" library (Shell Lightweight Utility Library). It has a lot of functionality; most importantly, it can alter files and folders.
We further investigated the MD5 and the SHA256 hash with some malware databases. Luckily, we found a match on the Website Virustotal [3]. The malware was first discovered in 2021 and apparently is hard to detect because

only 17/70 security vendors and no sandboxes flagged this file as malicious. Additionally, we can see all the imports needed, which confirms the previously seen part in the "additional information" upon the startup in Ghidra. The imports of the malware are displayed in the bottom table. We are not going to analyze each of them to keep the report concise because one can easily do it by oneself via Google.

KERNEL32.dll	**SHLWAPI.dll**
GetStdHandle	PathFindFileNameW
GetConsoleOutputCP	
EncodePointer	
DeleteCriticalSection	
GetCurrentProcess	
GetConsoleMode	
UnhandledExceptionFilter	
FreeEnvironmentStringsW	
InitializeSListHead	
GetLocaleInfoW	

Table 1: Imports [3]

Additionally, the information we found online due to the hashes of MD5 and SHA256 is that there is suspicion that the malware can delete files or directories, create a new process with a "PING.exe," observe specific registers for changes, open files with deletion access rights and many things more. It is worth noting that mYSCpPoHAih is suspected of using AES Encryption [2][1].
Now, let's move to the analysis in Ghidra.

2.2 Code analysing

Since we previously discovered that the malware is a Windows executable, and we do not see a WinMain function. We go to the entry file because it is the first function being called. From there on, we search for a function without parameters, usually located at the end of said function. After a few clicks, we found our entry-point function. Thus, we rename it to "int wWinMain(HINSTANCE hInstance, HINSTANCE hPrevInstance, PWSTR pCmdLine, int nCmdShow)" since this is the default in every Windows program. We do this to achieve higher viability in the decompiler window potentially.
From this point on, we start reverse engineering the code by renaming the functions and variables to gain a better comprehension of the code. The encryption of the malware works as follows: the plaintext (seen as hexdump of a file) is split into 1008 bytes blocks, whereby each block is loaded into a buffer and then encrypted using AES-128-CBC, that is the 1008 bytes blocks are itself divided into 16 bytes blocks for the AES and chained using CBC mode. After that, each ciphertext block of 1008 bytes is prefixed with the IV (16 bytes) and the length of the plaintext block (4 bytes). The process is shown in Figure 3.

```
int wWinMain(HINSTANCE hInstance,HINSTANCE hPrevInstance,PWSTR pCmdLine,int nCmdShow)

{
  int iVar1;
  undefined auStack584 [32];
  WCHAR local_228 [264];
  ulonglong local_18;

  local_18 = DAT_140086038 ^ (ulonglong)auStack584;
  printf((char *)L"Getting current directory.\n");
  GetCurrentDirectoryW(0x104,local_228);
  thunk_FUN_140007590();
  Sleep(10000);
  thunk_FUN_14000f50();
  iVar1 = thunk_FUN_14000ae80(local_18 ^ (ulonglong)auStack584);
  return iVar1;
}
```

Figure 2: Entry-point function

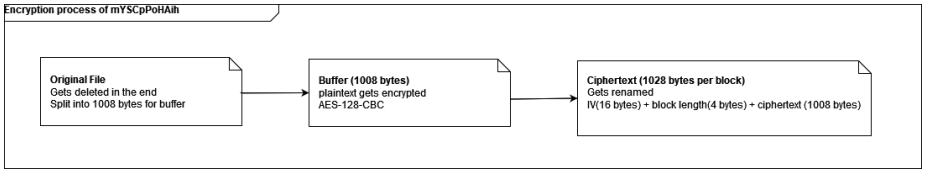

Figure 3: Encryption process

3 Targeted Files and Directories

The ransomware initially gets the path to the current directory. Then, it checks whether the received paths are valid (do not exceed the length of 260). If the input is correct, it proceeds elsewhere. It does nothing. Then, it searches for the first file in the current directory and checks if it has found something and if it is a file. After that, it checks that it is not a temporary file starting with "en". Then, the malware encrypts the file and deletes the original file. Later, it starts to search for the following file until no file is left.

Later, the whole above-described functionality gets executed again, but this time with the search on temporary that begins with "en" and renames them. In the very end, it closes the files.

Nevertheless, as one can see in figure 11, mYSCpPoHAih also does something else within the C directory. It pings the local machine with a loopback address (127.1.247.73) and deletes files. We can also see that the method GetModuleFileNameW uses an hModule, which is the base address for DLLs. This leads to the conclusion that it deletes DLL files. This can be verified in the figure 11. Upon that, it uses the CloseHandle to remove the object passed as a parameter from the system once the last object of it is closed. In summary, it deletes itself from the system once it has done its job.

4 Recovery Key & Tool

The masterkey to compute the roundkeys and IV for the AES encryption can be seen in Figures 4 and 5.
The recovery tool is written in Python and works by reversing the whole encryption process shown in Figure 3 as follows: The encrypted file is read as a bytestream and then split into 1028 bytes blocks in form IV (16 bytes) + plaintext length (4 bytes) + ciphertext block (1008 bytes), the first 20 bytes are then cut off respectively and the 1008 bytes ciphertext block decrypted to receive the corresponding plaintext block of 1008 bytes. After that, all plaintext blocks are concatenated into one plaintext and written into a new file as bytestream. The decryption is applied to all encrypted files within the directory of the Python script.

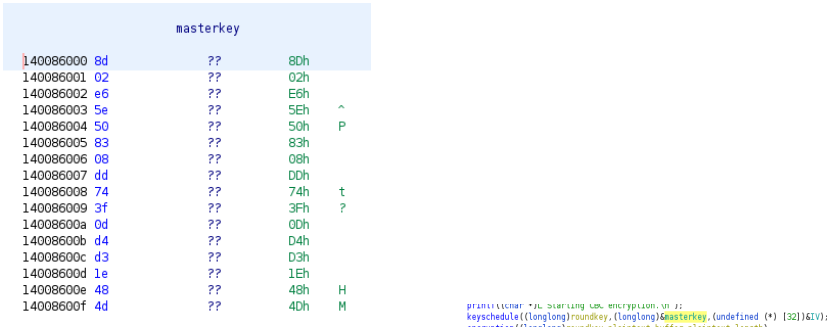

Figure 4: Masterkey used for AES to compute roundkeys

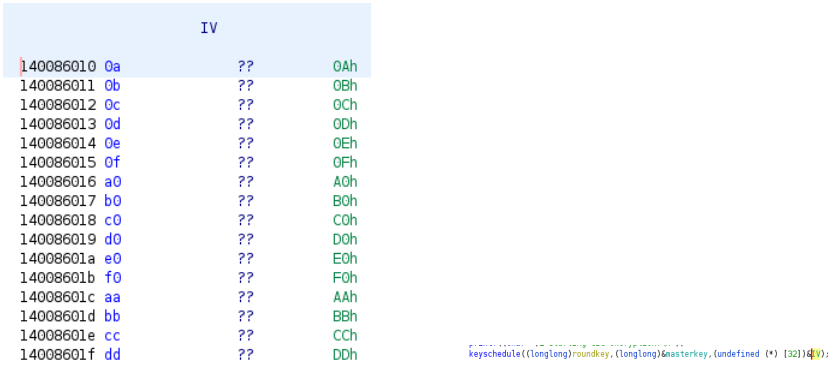

Figure 5: IV used for the AES

References

[1] Sandbox Falcon. *Free Automated Malware Analysis Service - powered by Falcon Sandbox - Search results.* URL: `https://www.hybrid-analysis.com/search?query=58d67e89e24ed64238f44b1295b03a4e54dfdabcbc53534d573ec496cac4434d` (visited on 03/14/2024).

[2] Joe Sandbox. *Automated Malware Analysis Report for mYSCpPoHAih - Generated by Joe Sandbox.* URL: `https://www.joesandbox.com/analysis/415666/0/html` (visited on 03/14/2024).

[3] VirusTotal. *VirusTotal - File - 58d67e89e24ed64238f44b1295b03a4e54dfdabcbc53534d573ec496cac4434d.* URL: `https://www.virustotal.com/gui/file/58d67e89e24ed64238f44b1295b03a4e54dfdabcbc53534d573ec496cac4434d/summary` (visited on 03/14/2024).

A Appendix

```
                    /* param_1: Current Directory Path */
curr_path_length_or_0(current_dir_path,260,(short *)&DAT_140070f64);
return_pointer_to_0(pointer_curr_dir_path,260,current_dir_path);
curr_path_length_or_0(pointer_curr_dir_path,260,(short *)&DAT_140070f68);
                    /* find filename! */
first_file = FindFirstFileW(pointer_curr_dir_path,(LPWIN32_FIND_DATAW)pointer_to_searched_filename
                           );
if (first_file == (HANDLE)0xffffffffffffffff) {
  DVar1 = GetLastError();
  errorHandler(L"No file found\n",DVar1);
}
else {
  printf((char *)L"Listing files in directory...\n");
  do {
                    /* Chek if finding is a file */
    if ((pointer_to_searched_filename._0_4_ & 0x10) == 0) {
      return_pointer_to_0(local_a68,260,current_dir_path);
      local_1100 = local_a68;
      curr_path_length_or_0(local_1100,260,local_10bc);
      GetModuleFileNameW((HMODULE)0x0,local_858,260);
      thunk_FUN_14000c700((undefined (*) [32])buffer,(undefined (*) [32])local_10bc,6);
                    /* returns zero if they are the sane */
      iVar2 = wcscmp(buffer,L"~en");
      if (iVar2 != 0) {
        _Str2 = PathFindFileNameW(local_858);
        iVar2 = wcscmp(local_10bc,_Str2);
        if (iVar2 != 0) {
          return_pointer_to_0(local_648,260,current_dir_path);
          local_1108 = local_648;
          curr_path_length_or_0(local_1108,260,(short *)&DAT_140070fd8);
          curr_path_length_or_0(local_1108,260,local_10bc);
          encryption(local_1100,local_1108);
          DeleteFileW(local_1100);
        }
      }
    }
    BVar3 = FindNextFileW(first_file,(LPWIN32_FIND_DATAW)pointer_to_searched_filename);
  } while (BVar3 != 0);
```

Figure 6: Path behaviour

```
GetModuleFileNameW((HMODULE)0x0,local_860,0x104);
thunk_FUN_14000c700((undefined (*) [32])local_ea0,(undefined (*) [32])local_10c4,6);
iVar2 = wcscmp(local_ea0,L"~en");
if (iVar2 != 0) {
  _Str2 = PathFindFileNameW(local_860);
  iVar2 = wcscmp(local_10c4,_Str2);
  if (iVar2 != 0) {
    thunk_FUN_140007bc0(local_650,0x104,in_stack_00000000);
    local_1110 = local_650;
    thunk_FUN_140007b20(local_1110,0x104,(short *)&DAT_140070fd8);
    thunk_FUN_140007b20(local_1110,0x104,local_10c4);
    encryption_process(local_1108,local_1110);
    DeleteFileW(local_1108);
  }
```

Figure 7: Delete file after encryption

```
printf((char *)L"Starting encryption.\n");
plaintext_file = CreateFileW(param_1,1,1,(LPSECURITY_ATTRIBUTES)0x0,3,0x80,(HANDLE)0x0);
if (plaintext_file == (HANDLE)0xffffffffffffffff) {
  DVar2 = GetLastError();
  thunk_FUN_140007510(L"Error opening plaintext file!\n",DVar2);
}
else {
  printf((char *)L"Successfully opened plaintext file, %s. \n",param_1);
  ciphertext_file = CreateFileW(param_2,2,1,(LPSECURITY_ATTRIBUTES)0x0,4,0x80,(HANDLE)0x0);
  if (ciphertext_file == (HANDLE)0xffffffffffffffff) {
    DVar2 = GetLastError();
    thunk_FUN_140007510(L"Error opening destination file!\n",DVar2);
  }
  else {
    printf((char *)L"Successfully created destination file, %s. \n",param_2);
    local_e8 = 992;
    plaintext_length = 0x3f0;
    plaintext_buffer = (undefined (*) [32])_malloc_base(0x3f0);
    if (plaintext_buffer == (undefined (*) [32])0x0) {
      thunk_FUN_140007510(L"Not enough memory to allocate file buffer. \n",0x8007000e);
    }
```

Figure 8: Encryption process 1

```
else {
  printf((char *)L"%i file buffer has been allocated. \n",(ulonglong)plaintext_length);
  actual_plaintext_length = (uint *)0x0;
  actual_plaintext_length = (uint *)_malloc_base(4);
  break_while_loop = '\0';
  do {
    BVar1 = WriteFile(ciphertext_file,&IV,16,numberOfBytesWritten,(LPOVERLAPPED)0x0);
    if (BVar1 == 0) {
      DVar2 = GetLastError();
      thunk_FUN_140007510(L"Error writing padding size.\n",DVar2);
      goto LAB_1400073da;
    }
    printf((char *)L"IV successfully added to file.\n");
    BVar1 = ReadFile(plaintext_file,plaintext_buffer,plaintext_length,numberOfBytesWritten,
                     (LPOVERLAPPED)0x0);
    if (BVar1 == 0) {
      DVar2 = GetLastError();
      thunk_FUN_140007510(L"Error reading plaintext!\n",DVar2);
      goto LAB_1400073da;
    }
```

Figure 9: Encryption process 2

```
if (numberOfBytesWritten[0] < plaintext_length) {
  break_while_loop = '\x01';
}
*actual_plaintext_length = numberOfBytesWritten[0];
BVar1 = WriteFile(ciphertext_file,actual_plaintext_length,4,numberOfBytesWritten,
                  (LPOVERLAPPED)0x0);
if (BVar1 == 0) {
  DVar2 = GetLastError();
  thunk_FUN_140007510(L"Error writing padding size.\n",DVar2);
  goto LAB_1400073da;
}
printf((char *)L"Length of file buffer successfully added to file.\n");
printf((char *)L"Starting CBC encryption.\n");
keyschedule((longlong)roundkey,(longlong)&masterkey,(undefined (*) [32])&IV);
encryption((longlong)roundkey,plaintext_buffer,plaintext_length);
printf((char *)L"Successfully encrypted file buffer. Writing to destination file...\n");
BVar1 = WriteFile(ciphertext_file,plaintext_buffer,plaintext_length,numberOfBytesWritten,
                  (LPOVERLAPPED)0x0);
```

Figure 10: Encryption process 3

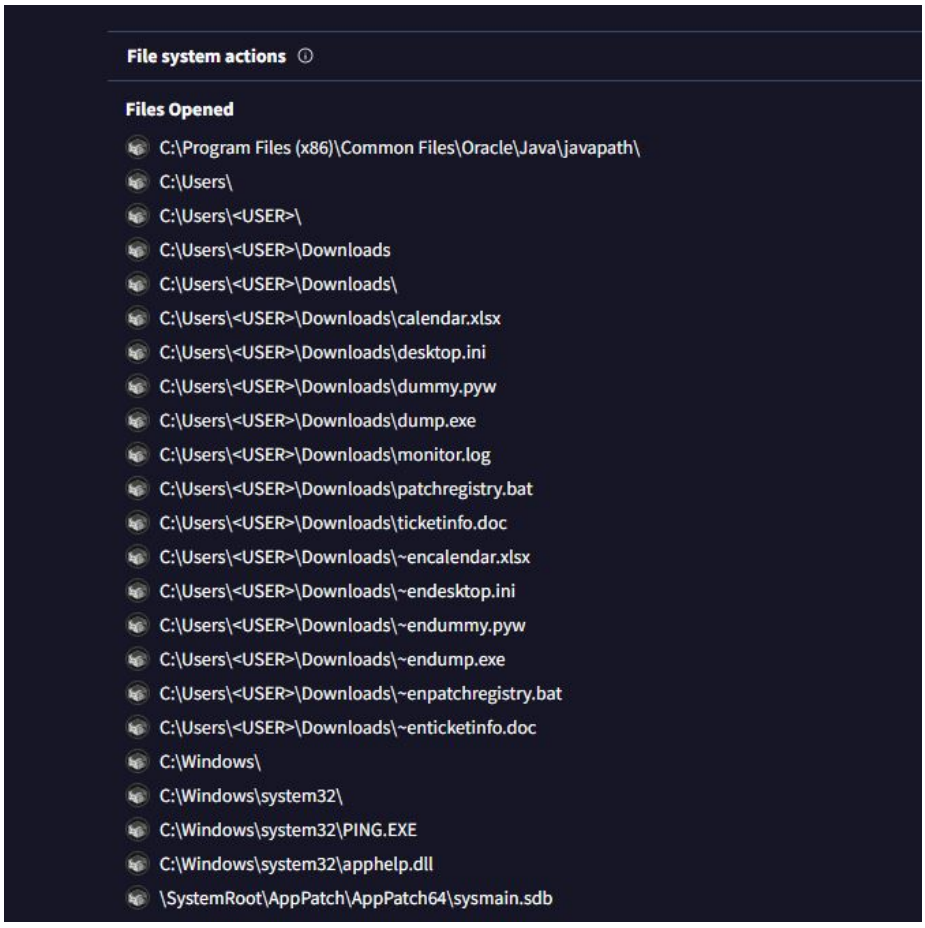

Figure 11: Files opened by the malware

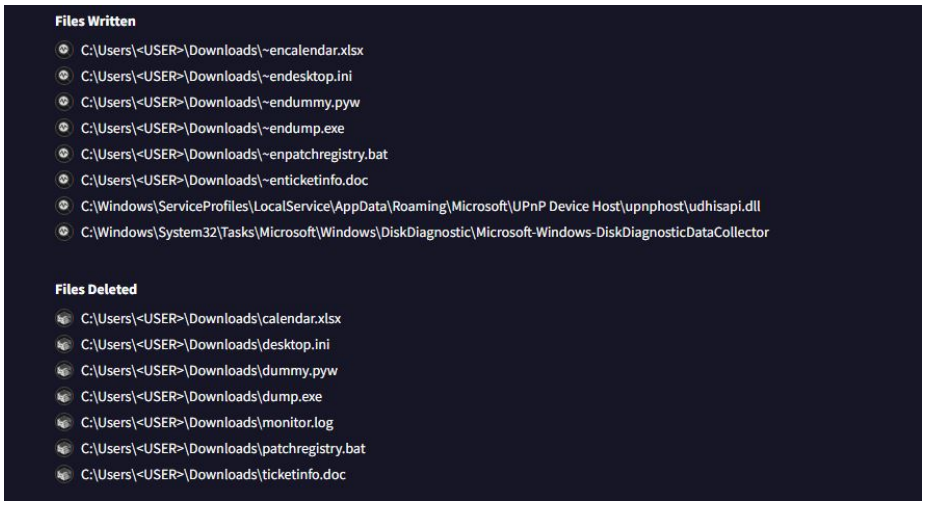

Figure 12: Files written and deleted by the malware

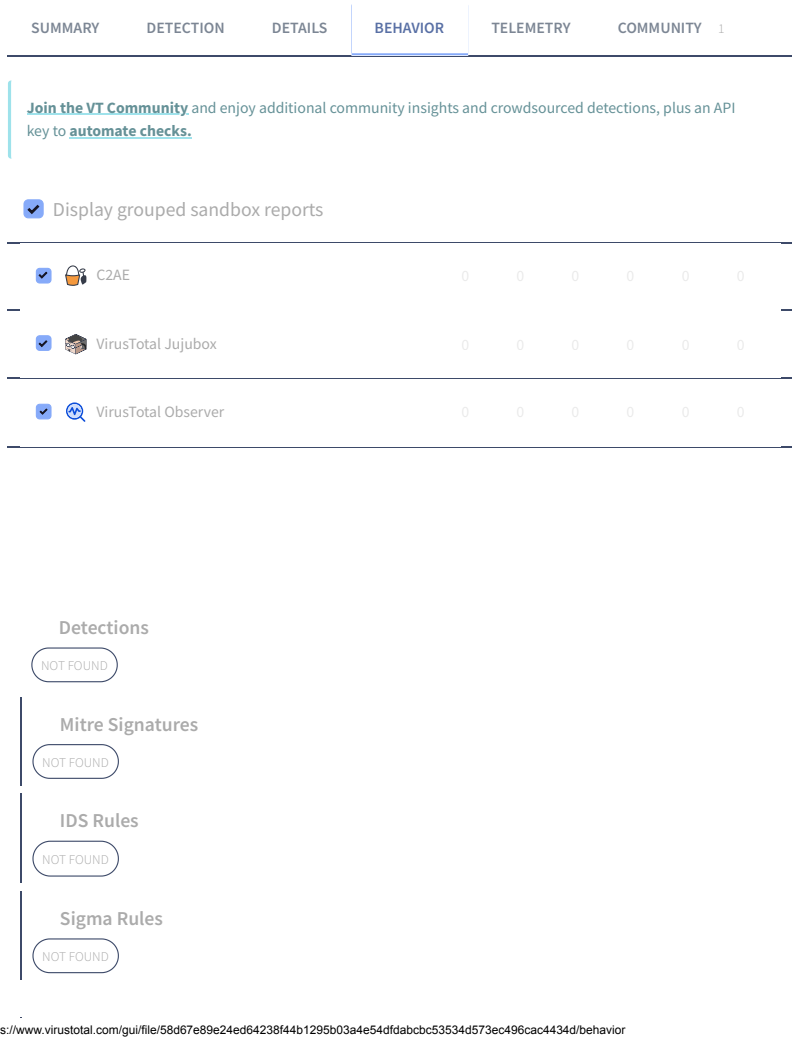

Figure 13: Virustotal - malware behaviour

Figure 14: Virustotal - malware details

URL, IP address, domain, or file hash

VIRUSTOTAL

Join the VT Community and enjoy additional community insights and crowdsourced detections, plus an API key to **automate checks.**

Security vendors' analysis	Do you want to automate checks?
Alibaba	Trojan:Win32/Udochka.f8b1f51f
Avast	Win64:Trojan-gen
AVG	Win64:Trojan-gen
Cynet	Malicious (score: 100)
ESET-NOD32	A Variant Of Generik.HZRMHTF
Fortinet	W32/PossibleThreat
GData	Win32.Trojan-Ransom.Filecoder.RRBF7V@gen
Gridinsoft (no cloud)	Ransom.Win64.Ransom.sa
Ikarus	Trojan.SuspectCRC
Kaspersky	Trojan.Win32.Udochka.aeg
McAfee	Artemis!1046F940C0CB
McAfee-GW-Edition	Artemis
Microsoft	Ransom:MacOS/Filecoder
Rising	Ransom.Filecoder!8.55A8 (CLOUD)
Sophos	Mal/Generic-S
Symantec	OSX.Trojan.Gen
TrendMicro-HouseCall	TROJ_GEN.R002H0DEK21
Acronis (Static ML)	Undetected
Ad-Aware	Undetected
AegisLab	Undetected
AhnLab-V3	Undetected
ALYac	Undetected
Antiy-AVL	Undetected
Arcabit	Undetected
Avira (no cloud)	Undetected
Baidu	Undetected
BitDefender	Undetected
BitDefenderTheta	Undetected

Figure 15: Virustotal - antivirus detection

```
int wWinMain(HINSTANCE hInstance,HINSTANCE hPrevInstance,PWSTR pCmdLine,int nCmdShow)

{
  int iVar1;
  undefined auStack584 [32];
  WCHAR local_228 [264];
  ulonglong local_18;

  local_18 = DAT_140086038 ^ (ulonglong)auStack584;
  printf((char *)L"Getting current directory.\n");
  GetCurrentDirectoryW(0x104,local_228);
  thunk_FUN_140007590();
  Sleep(10000);
  thunk_FUN_140006f50();
  iVar1 = thunk_FUN_14000ae80(local_18 ^ (ulonglong)auStack584);
  return iVar1;
}
```

Figure 16: Entrypoint function of the malware

Figure 17: Decryption of one of the encrypted images as proof